
WorldDeciding

Design and Development Presentation

Architecture, data model, implementation modules, and user interface.

Design and Development of a Web Platform for Cross-Cultural Opinion Analysis

Prepared by: [Student Name] | May 1, 2026

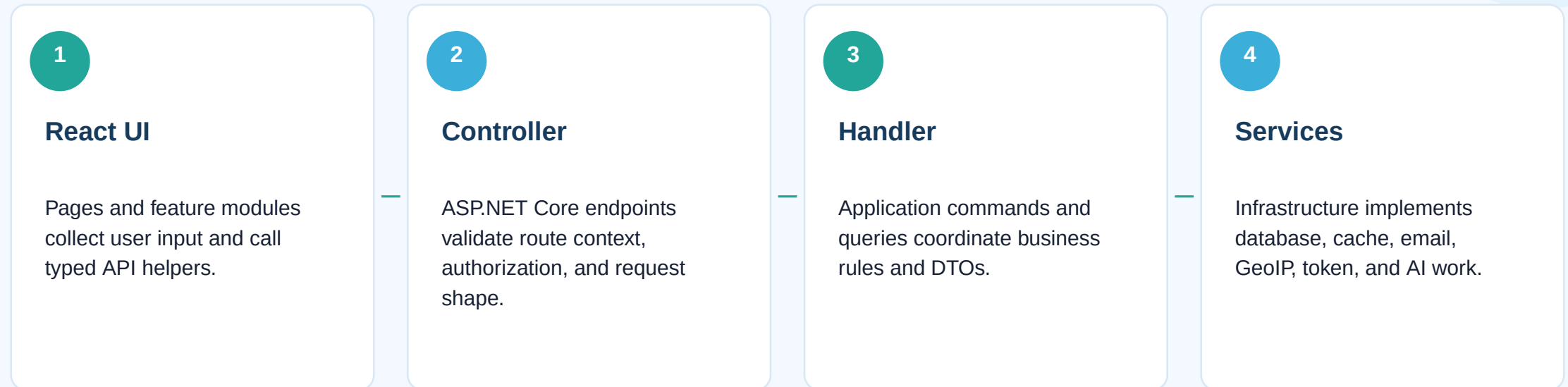
Agenda

- 1 Layered solution structure
- 2 Backend API modules and domain model
- 3 Voting, statistics, and country comparison implementation
- 4 Frontend experience and admin workflow
- 5 Security, privacy, and deployment design

Implemented Architecture

- Backend is split into Domain, Application, Infrastructure, and API projects.
- MediatR handlers implement command and query use cases.
- Infrastructure services implement EF Core, Redis, GeoIP, email, tokens, and AI.
- React frontend is organized by pages, entities, features, widgets, and shared modules.

Request Flow



Backend API Modules

- Auth: register, login, email confirmation, reset password, refresh token, logout.
- Questions: list, detail, stats, view recording, AI discussion summary.
- Votes: authenticated vote submission and cooldown handling.
- Comments: list, create, replies, likes.
- Admin: categories, question lifecycle, bulk import, analytics support.

Data Model

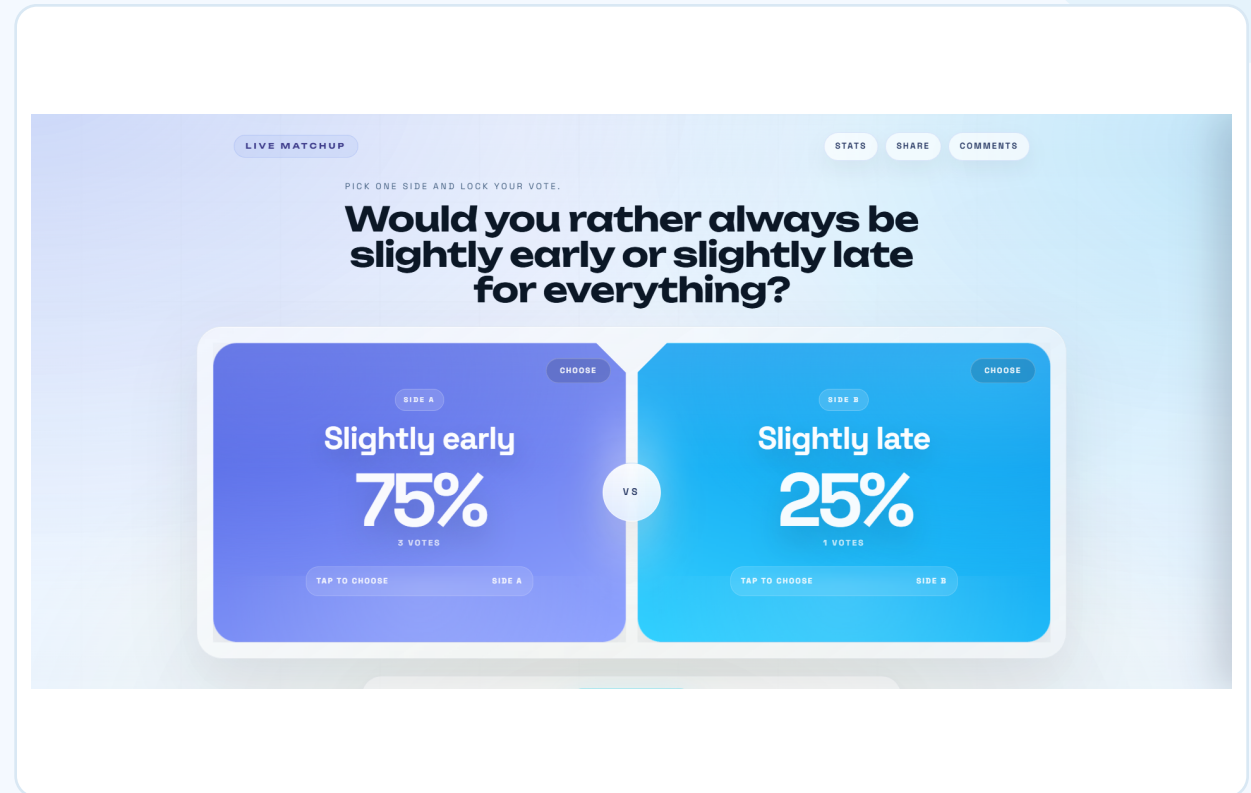
- Question and Option represent the voting prompt.
- Vote stores user, option, country metadata, confidence, and hashed IP.
- QuestionStatsDaily stores pre-aggregated daily vote/view counters.
- Comment, CommentLike, and QuestionCommentSummary support discussion and AI summaries.
- RefreshToken supports rotating token families and reuse detection.

Layer Responsibilities

Layer	Responsibility	Example Files
Domain	Entity definitions and identity models.	Question, Option, Vote, Comment, AppUser
Application	Commands, queries, DTOs, validators, interfaces.	CastVoteHandler, ListQuestionsHandler
Infrastructure	Persistence and external service implementations.	DbContext, Redis, GeoIP, AI, email
API	HTTP endpoints, middleware, auth policies, CORS.	Controllers, Program.cs, rate middleware

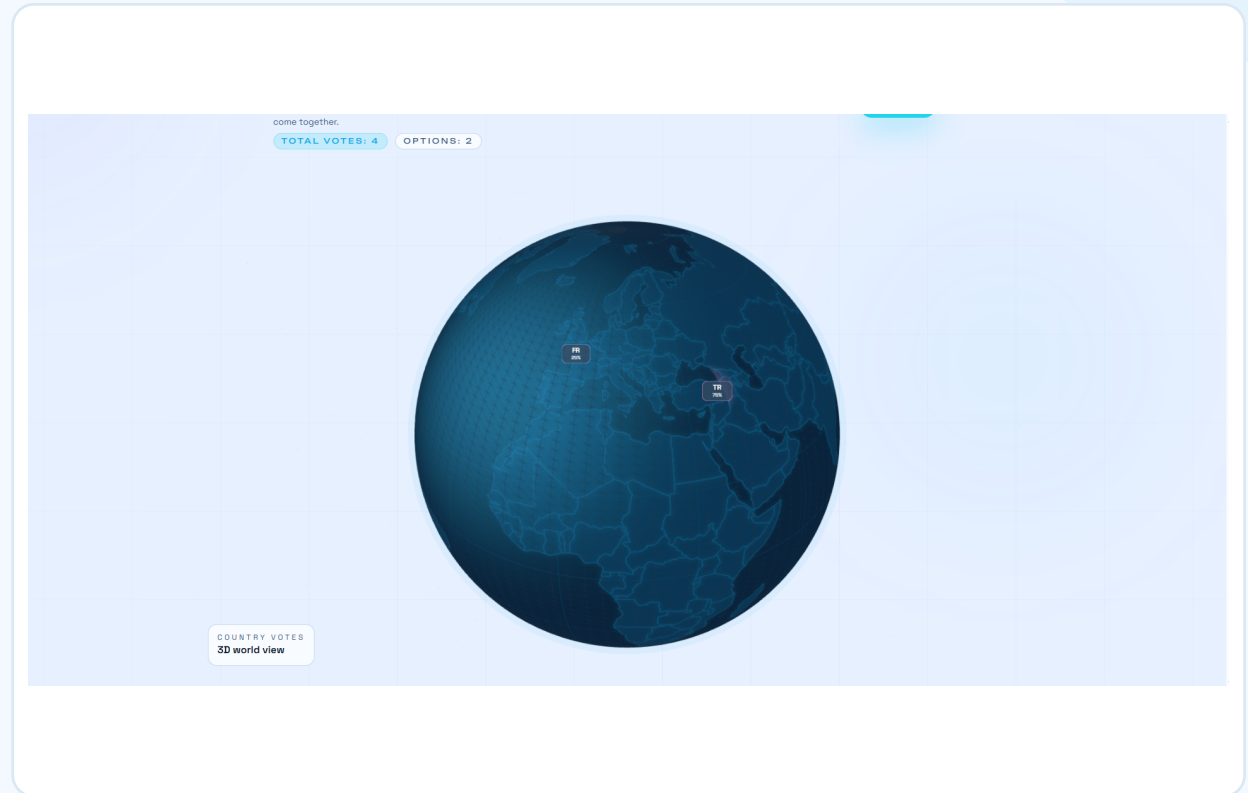
Development: Voting Implementation

- Validates option-question relationship.
- Extracts authenticated user ID from JWT claims.
- Hashes client IP before persistence.
- Uses Redis abuse checks for rapid vote attempts.
- Enforces one active vote per user and a 24-hour change cooldown.



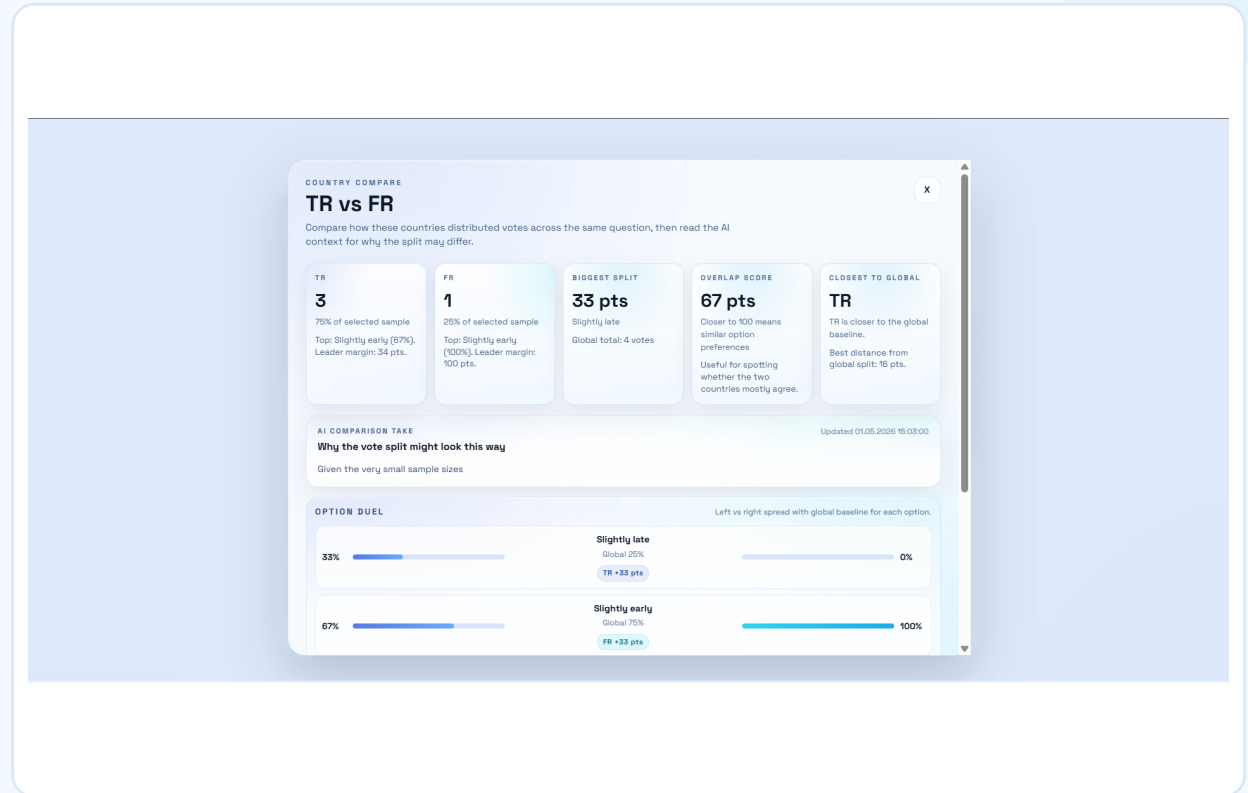
Development: Statistics Implementation

- Aggregates option counts and percentages.
- Groups votes by country for geographic analysis.
- Reads user demographics for gender and age-band breakdowns.
- Caches detailed stats briefly and maintains daily counters for leaderboards.



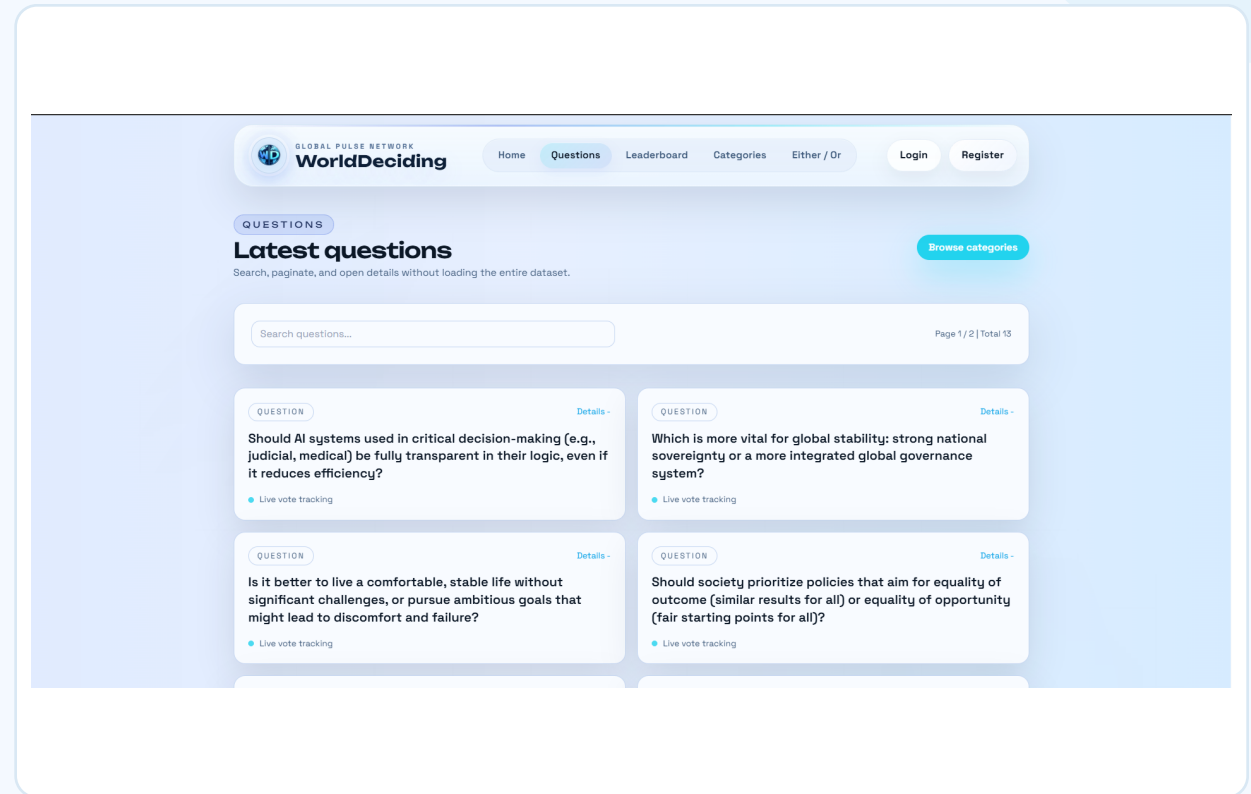
Development: Country Comparison

- Compares two selected ISO country codes.
- Shows country totals, option splits, global baseline, biggest gap, overlap score, and closest-to-global signal.
- Adds AI-generated tentative explanation with fallback text.
- Future improvement: suppress small samples by threshold.



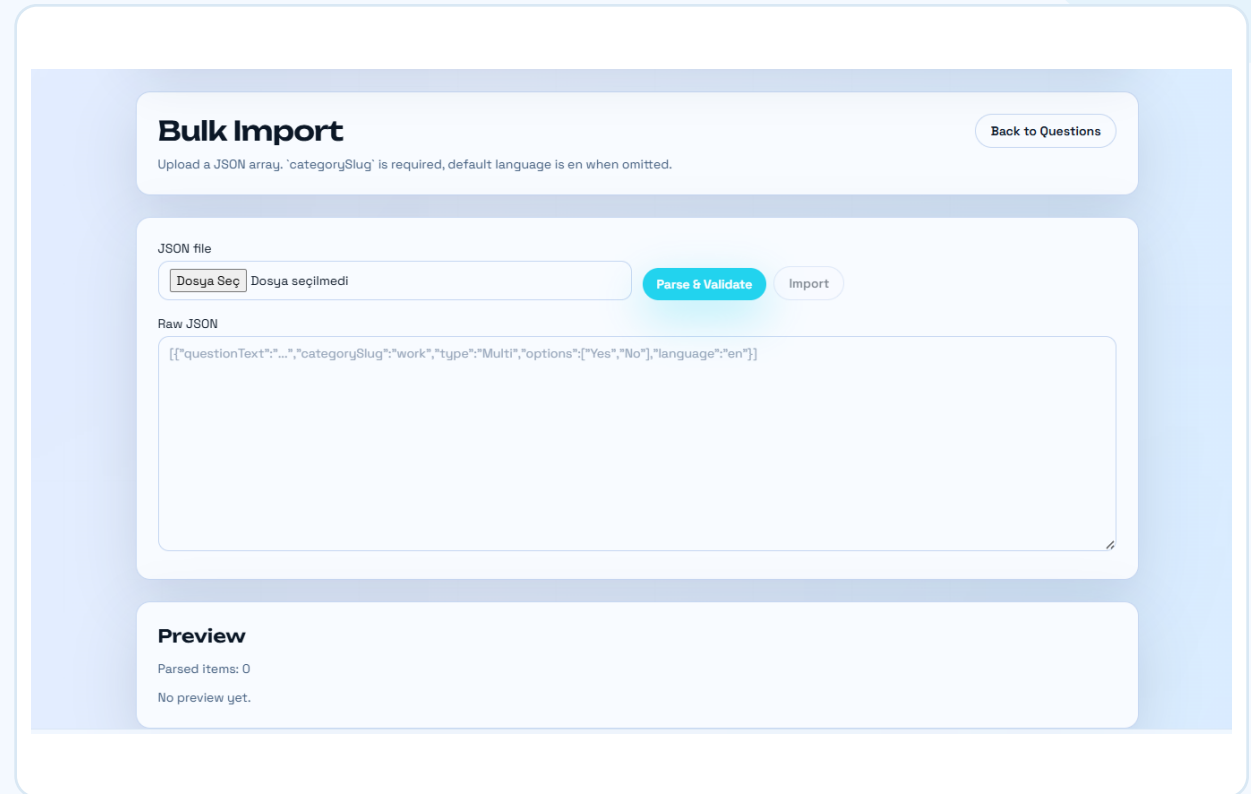
Development: Frontend Experience

- Home page shows live question, totals, categories, and trending questions.
- Question detail supports voting, comments, AI summaries, and navigation.
- Statistics page uses interactive country selection and visual breakdowns.
- Profile, leaderboard, category, login, register, privacy, and cookie pages complete the app flow.



Development: Admin Experience

- Admin dashboard centralizes content management.
- Question list filters by status, language, and search.
- Bulk import validates category, text length, options, language, and duplicates.
- Publish and archive endpoints manage the question lifecycle.



The screenshot displays the 'Bulk Import' section of an admin dashboard. At the top, there's a header with the title 'Bulk Import' and a 'Back to Questions' button. Below the header, a sub-header instructs the user to 'Upload a JSON array. 'categorySlug' is required, default language is en when omitted.' The main area is divided into two sections: 'JSON file' and 'Raw JSON'. The 'JSON file' section features a file upload button labeled 'Dosya Seç' (File Select) and a text input field showing 'Dosya seçilmedi' (File not selected). To the right of the input field are two buttons: 'Parse & Validate' (highlighted in blue) and 'Import'. The 'Raw JSON' section contains a large text area with a sample JSON array:

```
[{"questionText": "...", "categorySlug": "work", "type": "Multi", "options": ["Yes", "No"], "language": "en"}]
```

. At the bottom, a 'Preview' section shows 'Parsed items: 0' and 'No preview yet.'

Security and Privacy Design

Security Controls

- JWT validation and refresh token rotation
- Email confirmation before login
- Role-based admin authorization
- Redis-backed rate and abuse counters

Privacy Controls

- Salted IP hash instead of raw IP persistence
- Consent-based analytics activation
- Country confidence metadata
- Privacy and cookie pages in the frontend

Deployment Design

1

PostgreSQL

Durable relational store with EF Core migrations.

2

Redis

Cache and rate limiting for high-frequency actions.

3

API

ASP.NET Core container exposes the production HTTP service.

4

Frontend

Vite build is served by the web hosting layer.